Relatório Projeto 2

João Pedro Neves e Juan Pablo Tomba

20 de novembro de 2025

1 Introdução

Neste trabalho, iremos abordar diversos métodos de ordenação: BubbleSort, SelectionSort, InsertionSort, ShellShort, QuickSort (Usando mediana de 3 para o pivô), HeapSort, MergeSort, Contagem dos Menores e RadixSort.

O objetivo principal é analisar o comportamento empírico destes algoritmos em relação à análise assintótica teórica.

A análise de desempenho baseou-se em três métricas fundamentais exigidas na especificação do projeto:

- O número de comparações entre chaves;
- O número de movimentações (trocas ou alocações) de registros;
- O tempo total de execução do algoritmo.

2 Implementação

Os algoritmos foram implementados na linguagem C, utilizando uma estrutura de dados **Estatística** no formato de uma struct para armazenar os valores das variáveis comparações, movimentações e trocas para cada algoritmo, além de funções de medição de tempo para aferir o desempenho computacional.

```c
typedef struct estatistica_ Est;

struct estatistica_ {
    unsigned int comparacoes;
    unsigned int movimentacoes;
    double tempo;
};
```

Todos os algoritmos foram implementados de forma similar as implementações apresentadas em sala de aula e nos materiais disponibilizados pelo professor.

Para garantir a validade estatística dos resultados e cumprir os requisitos do projeto, foram adotados os seguintes procedimentos de teste:

2.1 Cenários de Teste

Os métodos foram submetidos a vetores com quatro ordens de grandeza distintas: **100, 1.000, 10.000 e 100.000 elementos**. Para cada um destes tamanhos, foram gerados três tipos de disposição de dados:

- **Ordenados:** Vetores já dispostos em ordem crescente, simulando o melhor caso para alguns algoritmos (como o Insertion Sort) e verificando a estabilidade de outros.
- **Inversamente Ordenados:** Vetores dispostos em ordem decrescente, simulando o pior caso teórico para algoritmos quadráticos básicos.
- **Aleatórios:** Vetores preenchidos com números inteiros gerados de forma aleatória.

Os vetores ordenados tanto na ordem normal quanto inversamente foram gerados com os respectivos números do intervalo. Um vetor da ordem de 100 elementos ordenado inversamente foi gerado com os valores 99, 98, 97, ..., 0 por exemplo.

```c
int* montaOrdenado(int n){
    int* vetor = (int*)malloc(n * sizeof(int));
    for (int i = 0; i < n; i++)
        vetor[i] = i;

    return vetor;
}
```

Os aleatórios, no entanto, utilizaram um esquema diferente. Cada algoritmo rodou para 5 vetores diferentes gerados utilizando a função rand() da linguagem C, e usando a seed como sendo o tempo do sistema em segundos a partir da função time(NULL) da biblioteca "time.h".

Ao rodar os 9 algoritmos, os dados foram sendo salvos em um array de structs estatistica respectivo aos algoritmos que estavam rodando. Por fim, ele percorre cada uma das structs geradas, pega os valores ali salvos e faz a média do valor total pela quantidade de execuções, nesse caso 5.

```c
int main(){
    int tamanhos[4] = {100, 1000, 10000, 100000};

    FILE* arq = criarCSV("resultados.csv");
    // Seed base dinâmica para garantir que cada execução do programa gere um novo conjunto de vetores
    unsigned int baseSeed = (unsigned int)time(NULL);

    for (int i = 0; i < 4; i++){ // Loop para os tamanhos (Controlado por 'i')

        int* vetorOrdenado = montaOrdenado(tamanhos[i]);
        rodarTodos(vetorOrdenado, "Ordenado", tamanhos[i], arq);
        free(vetorOrdenado);

        int* vetorReversamenteOrdenado = montaReversamenteOrdenado(tamanhos[i]);
        rodarTodos(vetorReversamenteOrdenado, "Reverso", tamanhos[i], arq);
        free(vetorReversamenteOrdenado);


        int** aleatorios = (int**)malloc(sizeof(int*) * 5);
        // Geração dos 5 vetores aleatórios (Controlado por 'j')
        for (int j = 0; j < 5; j++){
            // Garante que a seed seja diferente para cada um dos 5 vetores e para cada tamanho 'i'
            unsigned int seed = baseSeed + (i * 10) + j;
```

```
            aleatorios[j] = montaAleatorio(tamanhos[i], seed);
        }

        rodarTodosAleatorio(aleatorios, tamanhos[i], arq);

        // Liberação de memória (Controlado por 'k')
        for (int k = 0; k < 5; k++){
            free(aleatorios[k]);
        }
        free(aleatorios);

        fflush(arq);

}
```

Por último, ao rodar cada algoritmo, obter o número de comparações, movimentações e o tempo de execução, o código salva esses dados em um arquivo csv com o formato:

algoritmo, tipo, tamanho, comparacoes, movimentacoes, tempo

```
void salvarCSV(FILE* f, const char* algoritmo, const char* tipo, int n, Est *e)
    {
    fprintf(f, "%s,%s,%d,%u,%u,%.10f\n",
        algoritmo,
        tipo,
        n,
        e->comparacoes,
        e->movimentacoes,
        e->tempo
    );
}
```

Exemplo do funcionamento da função rodarTodos com o BubbleSort:

```
void rodarTodos(int v[], const char* tipo, int n, FILE* arq){
    int* copia = (int*)malloc(sizeof(int) * n);
    Est est; // Struct alocada na pilha
    double tempo_gasto;

    // BubbleSort
    copiarVetor(v, copia, n);
    resetarEstatistica(&est);
    clock_gettime(CLOCK_MONOTONIC, &inicio);
    BubbleSort(copia, n, &est);
    clock_gettime(CLOCK_MONOTONIC, &fim);
    tempo_gasto = (fim.tv_sec - inicio.tv_sec) + (fim.tv_nsec - inicio.tv_nsec)
     / 1000000000.0;
    est.tempo = tempo_gasto;
    salvarCSV(arq, "Bubble", tipo, n, &est);
```

2.2 Detalhes Técnicos

- **QuickSort:** Foi implementada a estratégia de "Mediana de 3" para a escolha do pivô.
- **ShellSort:** Implementado com incrementos seguindo a sequência de N/2.

3 Resultados obtidos

O algoritmo salva as informações em um arquivo csv do tipo: algoritmo, tipo, tamanho, comparações, movimentações e tempo.

Com isso, plotamos os gráficos utilizando a biblioteca "pandas"em python, e transformamos o tempo em ms, para ficar visualmente melhor.

Como pedido no enunciado, tiramos a média de 5 execuções dos vetores aleatoriamente ordenados para medir o tempo de execução.

3.1 Vetor ordenado

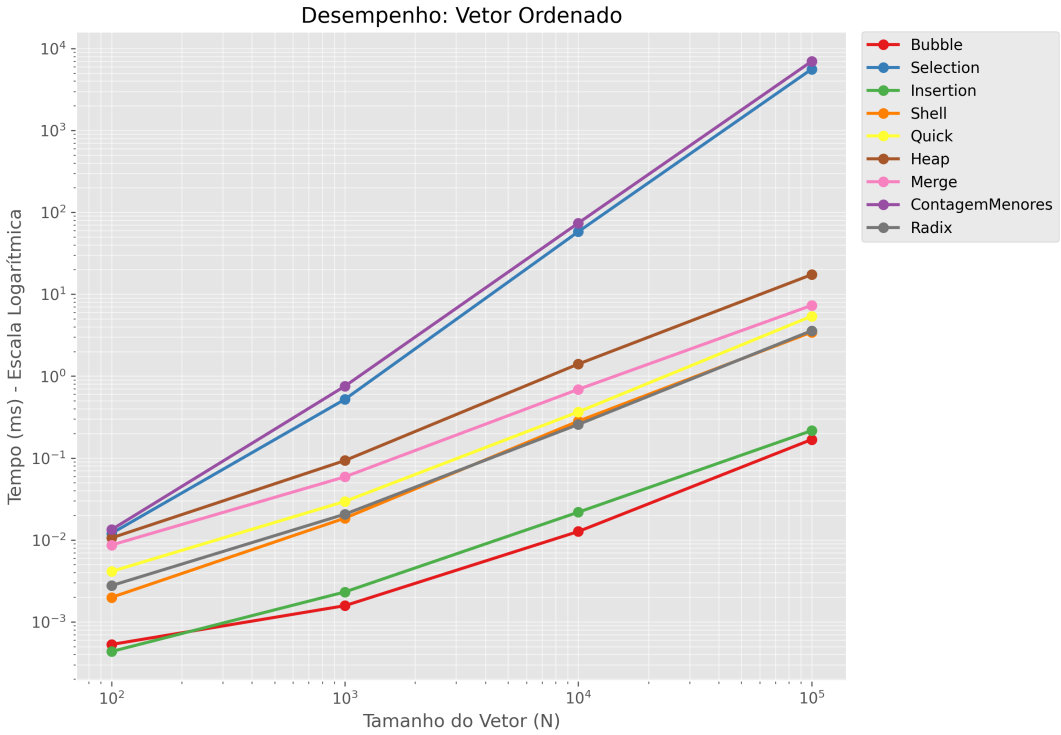

Gráfico de desempenho de tempo por tamanho n de um vetor ordenado

3.2 Vetor inversamente ordenado

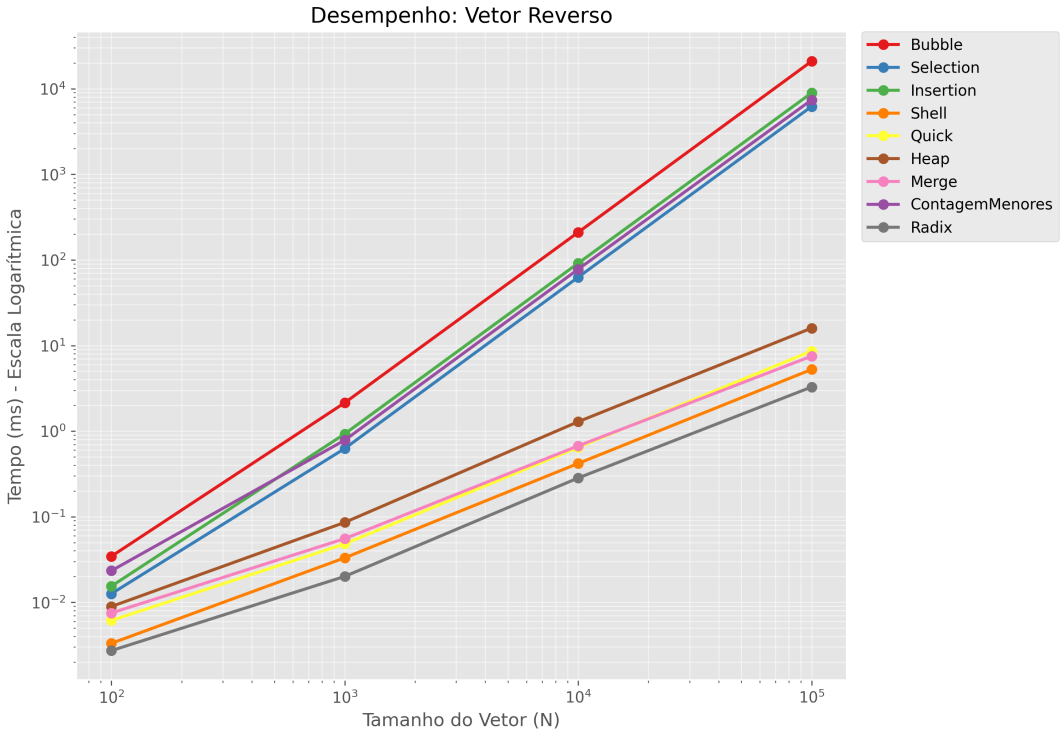

Gráfico de desempenho de tempo por tamanho n de um vetor inversamente ordenado

3.3 Vetor ordenado aleatoriamente

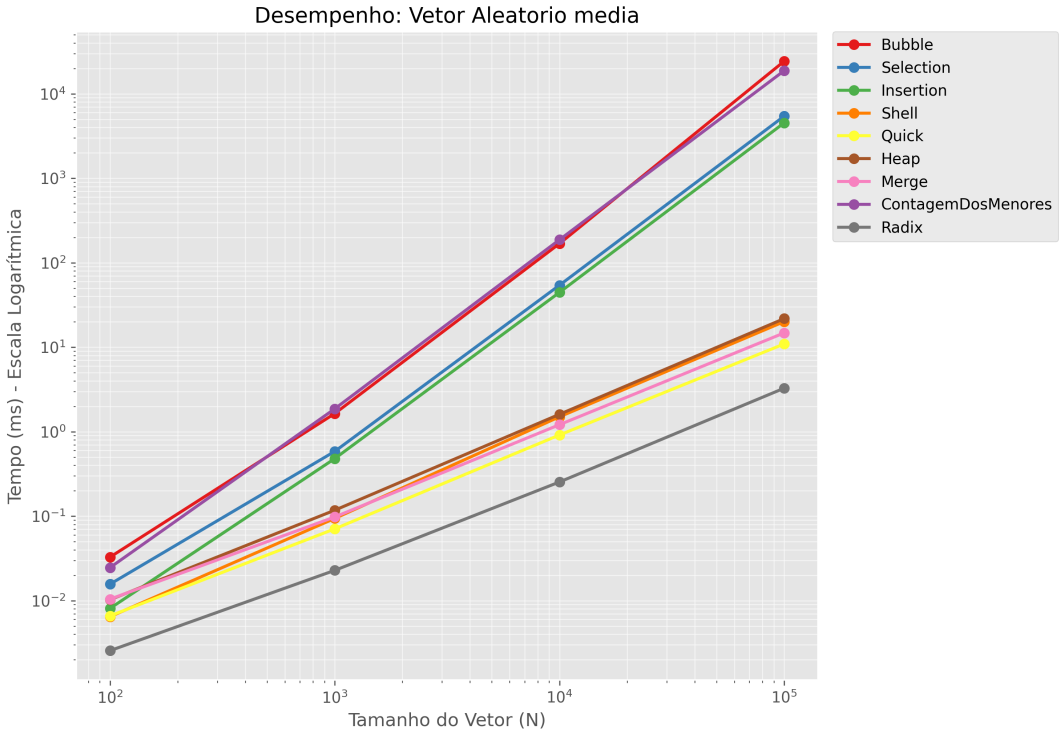

Gráfico de desempenho de tempo por tamanho n de um vetor aleatoriamente ordenado

4 Análise Teórica

Com base nas implementações, temos como deduzir as complexidades dos algoritmos, como mostrado na tabela a seguir:

Complexidade dos algoritmos

Algoritmo	**Melhor Caso**	**Caso Médio**	**Pior Caso**	**Espaço Extra**
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Shell Sort	$O(n \log n)$	$\sim O(n^{1.5})$	$O(n^2)$	$O(1)$
Quick Sort (mediana de 3)	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Contagem dos Menores	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(3n)$
Radix Sort	$O(d(n+k))$	$O(d(n+k))$	$O(d(n+k))$	$O(n+k)$

5 Análise Empírica

A análise experimental dos resultados permite confrontar a complexidade teórica com o tempo de execução real obtido nos testes com vetores de tamanho $N = 100, 1.000, 10.000$ e 100.000. A seguir, discutimos o desempenho dos algoritmos nos três cenários propostos.

5.1 Vetores Ordenados

Neste cenário, os resultados destacaram a diferença entre algoritmos adaptativos e não-adaptativos:

- **Melhor Desempenho ($O(N)$):** O *Insertion Sort* e o *Bubble Sort* apresentaram os tempos mais baixos. Isso ocorre devido à característica adaptativa desses algoritmos: o *Insertion* apenas percorre o vetor confirmando a ordem, e o *Bubble Otimizado*, utiliza a *flag* de troca para interromper a execução na primeira passagem.

- **Pior Desempenho ($O(n^2)$):** O **Contagem de Menores** e o **Selection Sort** apresentaram os piores tempos. Ambos são algoritmos não-adaptativos neste cenário. O *Selection Sort* continua realizando varreduras completas mesmo com o vetor ordenado. Similarmente, o *Contagem de Menores*, com sua complexidade quadrática $O(n^2)$, não tira proveito da ordenação prévia, resultando em tempos elevados idênticos aos do *Selection*.

5.2 Vetores Inversamente Ordenados

Este cenário representa o **pior caso** teórico para diversos métodos.

- **Piora de desempenho:** O *Bubble Sort* sofreu a maior penalidade, tornando-se o mais lento de todos devido ao número máximo de trocas e comparações. O *Insertion Sort* também degradou para $O(n^2)$, igualando-se ao desempenho consistentemente ruim do *Selection Sort* e do *Contagem de Menores* neste cenário.

- **Consistência Log-Linear:** O *Merge Sort*, *Heap Sort* e *QuickSort* (com pivô mediana de 3) mantiveram curvas de desempenho consistentes ($O(N \log N)$), provando duas complexidades fixas independente da disposição inicial dos dados.

5.3 Vetores Aleatórios (Caso Médio)

Sendo o cenário mais comum, a eficiência ficou clara:

- **Eficiência Linear:** O *Radix Sort* foi o mais rápido para todos os valores de N. Sua complexidade $O(d(n+k))$ permitiu superar os limites dos algoritmos baseados em comparação.

- **Desempenho Inferior:** O grupo formado pelo **Selection Sort**, **Bubble Sort** e **Contagem de Menores** apresentou os tempos mais altos. A natureza quadrática $O(n^2)$ desses algoritmos torna o processamento de grandes volumes de dados ($N = 100.000$) proibitivamente lento em comparação aos métodos log-lineares e ao Radix Sort que não utiliza comparações.

5.4 Conclusão

Respondendo às questões propostas:

1. **Qual algoritmo teve o melhor desempenho?** O **Radix Sort** apresentou o melhor tempo absoluto nos grandes vetores (10^5), seguido de perto pelo *QuickSort* e *Shell Sort*. Para o caso específico de vetores já ordenados, o *Insertion Sort* e *Bubble Sort Otimizado* são imbatíveis devido ao melhor caso $O(N)$.

2. **Qual teve o pior desempenho?** No cenário geral, os algoritmos quadráticos ficaram muito atrás. O **Bubble Sort** teve o pior desempenho global nos casos não-ordenados (Reverso e Aleatório). Já no cenário Ordenado, o **Selection Sort** e o **Contagem de Menores** foram os piores, pois não possuem mecanismos de adaptação para interromper o trabalho desnecessário.

3. **Considerações Finais:** A análise confirmou que a implementação utilizada do **Contagem de Menores** possui complexidade $O(n^2)$, comportando-se de maneira similar ao *Selection Sort*. Para uso geral em grandes conjuntos de dados, algoritmos como *QuickSort*,*Merge Sort* e *Heap Sort* oferecem o melhor equilíbrio, enquanto métodos elementares como *Bubble*, *Selection* e *Contagem* devem ser evitados.